

# Computational Investigations of Fermat Factorization with CRT, CSP, and Quadratic-Residue Sieving

Experimental Research Record

September 2026

## Abstract

This paper documents a sequence of computational experiments investigating odd semiprime factorization through Fermat’s identity, Chinese-remainder-theorem (CRT) constructions, constraint-satisfaction formulations, and quadratic-residue (QR) sieving. The work began with residue and carry decompositions and progressively moved toward a practical Fermat-search sieve. Several structural ideas were tested and rejected: determinant/rank defects were not universal, carry matrices were not universally singular, and important multiplicative relations in the derived  $C$ -representation were shown to be tautological because arbitrary residue vectors can generate consistent  $C$  arrays. In contrast, a QR sieve based on necessary conditions of the form  $x^2 - n \in QR(\ell)$  produced substantial end-to-end speedups on controlled Fermat intervals. Optimization experiments established that minimizing the survivor count is not equivalent to minimizing runtime; representation, materialization, and prime-prime interactions matter. Local 1-for-1 and 2-for-2 neighborhood search substantially improved practical performance, with measured speedups up to roughly  $716\times$  on the fixed benchmark suite. Controlled random instances at 66–88 bits repeatedly favored a common core of small QR primes, suggesting a recurring computational structure, but the random generator deliberately restricted factor separation and therefore did not establish uniform average-case behavior. A later experiment generated genuinely wide independent factors and showed why the naïve interval-materialization approach breaks down: the Fermat interval becomes astronomically large. The paper concludes with a precise account of what has been established, what has been ruled out, and what an appropriate next generalized QR experiment would require.

## 1 Introduction

Let

$$n = pq$$

be an odd semiprime with odd primes  $p < q$ . Fermat’s factorization method writes

$$x = \frac{p+q}{2}, \quad y = \frac{q-p}{2},$$

so that

$$x^2 - n = y^2. \tag{1}$$

The direct search begins at

$$x_0 = \lceil \sqrt{n} \rceil$$

and increments  $x$  until  $x^2 - n$  becomes a perfect square.

The computational research recorded here explored two broad questions. First, can arithmetic structure expressed through CRT, radix decompositions, carries, and constraint propagation expose the hidden factors more efficiently than direct search? Second, can the Fermat search

itself be filtered aggressively using modular necessary conditions, particularly quadratic residues modulo small primes?

The experiments produced both positive and negative results. A number of seemingly strong algebraic constraints turned out to encode tautological consequences of multiplication modulo the chosen bases. Conversely, QR filtering proved highly effective when the Fermat interval was small enough to be represented explicitly. The central lesson is therefore methodological: structural elegance is not sufficient; an experimentally useful invariant must survive independence tests, and an information-theoretically strong sieve must also be evaluated at end-to-end runtime and realistic memory cost.

## 2 Mathematical Setup

### 2.1 Fermat coordinates

From (1),

$$x^2 - n = y^2,$$

with

$$x = \frac{p+q}{2}, \quad y = \frac{q-p}{2}.$$

The width of the direct Fermat interval is therefore approximately

$$I = x - \lceil \sqrt{n} \rceil.$$

Ignoring the final rounding by one integer, an exact continuous identity is

$$\frac{p+q}{2} - \sqrt{pq} = \frac{(\sqrt{q} - \sqrt{p})^2}{2}. \quad (2)$$

Writing  $r = q/p$ , one obtains

$$\frac{\frac{p+q}{2} - \sqrt{pq}}{\sqrt{pq}} = \frac{r+1-2\sqrt{r}}{2\sqrt{r}}. \quad (3)$$

Thus Fermat's practical difficulty is controlled primarily by factor separation rather than by bit length alone.

### 2.2 Quadratic-residue filtering

For any odd prime  $\ell$ , the true Fermat value satisfies

$$x^2 - n = y^2.$$

Therefore

$$x^2 - n \pmod{\ell}$$

must belong to the set of quadratic residues modulo  $\ell$ . Define

$$M_\ell(x) = \begin{cases} 1, & x^2 - n \in QR(\ell), \\ 0, & \text{otherwise.} \end{cases}$$

For a selected prime set  $S$ ,

$$M_S(x) = \prod_{\ell \in S} M_\ell(x). \quad (4)$$

The true factorization is never removed by this sieve. If the interval is explicit, the remaining candidates are those  $x$  for which every selected modular condition passes.

For odd  $n$ , the prime 2 is uninformative because

$$x^2 - n \equiv 0 \pmod{2},$$

so 2 was excluded from the useful QR pool.

### 3 Experimental Program

Every experiment in the research record was explicitly delimited by the markers

START EXPERIMENT N

and

FINISHED EXPERIMENT N

so that benchmark runs could be distinguished and compared.

## 4 CRT, Radix, and Constraint-Satisfaction Experiments

### 4.1 Early algorithmic comparison

Experiment 123 compared Pollard-rho with direct prime scanning in the roughly 30–53 bit range. Pollard-rho was consistently preferable at these sizes, establishing that direct trial division should not be treated as the sole baseline.

Experiment 125 examined multi-radix CRT filtering. The effect was essentially a constant-factor sieve rather than a qualitative change in the search problem.

Experiment 129 established an important distinction: once the combined CRT modulus exceeds  $\sqrt{n}$ , a residue vector is unique, but the computational problem becomes finding the correct vector. CRT uniqueness therefore does not itself constitute a factorization algorithm.

### 4.2 Rank, carry, and matrix structure

Experiment 132 produced a determinant formulation of the form

$$\det(Q - E) = 0,$$

while the empirical rank behavior did not support the hypothesis that  $E$  is generically low rank. Experiment 154 tested whether a derived carry matrix is universally singular and found counterexamples. Consequently, neither a universal determinant defect nor universal carry singularity was available as a general invariant.

### 4.3 Global CSP and multiplicative structure

Experiment 158 applied AC-3 style propagation to a global radix-residue CSP. Arc consistency was useful but left broad domains. Experiments 162–166 then exposed a deeper structural simplification. In the tested construction,

$$C_{r,s}(a, b) = F_{n,r,s}(ab), \tag{5}$$

so the apparent two-variable dependence was mediated by the product  $ab$ .

Experiments 167–177B investigated increasingly strong local consistency. Rare-cell anchoring, rectangle constraints, MRV search, partial-side methods, and CRT lifting could recover factors on controlled examples. In particular, rare-cell anchoring plus AC-3 plus CRT gave measured times around 0.126 s at 30 bits, 3.17 s at 36 bits, and 5.32 s at 42 bits, while the 48-bit instance became too slow. A CRT parameterization (Experiment 168) produced approximately 0.019 s, 0.303 s, 3.287 s, 3.143 s, and 1.270 s at 30, 36, 42, 48, and 54 bits respectively.

The decisive negative result came from Experiments 179–180. Arbitrary residue vectors  $A$  and  $B$  can generate a consistent  $C$  array. Hence a large class of apparent multiplicative relations in  $C$  is tautological. This explains why increasingly elaborate closure rules could enforce consistency without necessarily revealing factor-specific information.

Experiment 181 showed poor scaling for pure residue search. Experiment 182 recovered factors using residue information together with an interval/quotient constraint, but a 54-bit case took about 54 s. Experiment 183 formalized the remaining residue freedom and clarified that residue data alone is insufficient.

## 5 Fermat Optimization

Experiment 185 demonstrated that segmented trial division produced roughly  $1.27\text{--}1.35\times$  speedups, useful as an implementation optimization but not a structural breakthrough.

Experiment 186B tested modular Fermat search. Representative results are shown in Table 1.

Table 1: Representative modular-Fermat measurements.

| Bits | Ordinary iterations | Ordinary time | Filtered iterations | Filtered time |
|------|---------------------|---------------|---------------------|---------------|
| 30   | 7,845               | 0.00227 s     | 813                 | 0.00362 s     |
| 36   | 64,997              | 0.0213 s      | 2,533               | 0.00523 s     |
| 42   | 523,271             | 0.178 s       | 48,928              | 0.201 s       |
| 48   | 4,191,864           | 1.40 s        | 326,639             | 1.29 s        |
| 54   | capped at 20M       | 6.55 s        | 3.83M               | 3.86 s        |
| 54   | capped at 20M       | 6.55 s        | 2.09M               | 6.07 s        |

The key lesson was that arithmetic filtering reduces the number of candidate positions but introduces overhead. Candidate count and runtime must therefore be measured separately.

## 6 Development of the QR Sieve

### 6.1 Correct search variable and density

Experiments 190–191 developed a segmented QR sieve. The variable was initially implemented incorrectly using  $y$  and subsequently corrected to the actual Fermat search variable  $x$ . After correction, empirical survivor densities were consistent with the expected product of local QR densities, supporting the interpretation of the sieve as a composition of modular necessary conditions.

### 6.2 Prime selection is a runtime optimization problem

Experiments 192–196 showed that the prime that maximizes marginal survivor reduction is not always the prime that maximizes end-to-end speed. This distinction became central to later optimization.

Experiments 198–204 examined bit-packed masks, cost models, and memory behavior. Bit packing can make pure set intersection extremely fast, yet materialization and representation conversion can dominate actual runtime. A useful cost ranking was obtained, but the absolute runtime model was not accurate enough to serve as a predictive model.

### 6.3 Greedy and local search

Experiment 205 used adaptive marginal-gain greedy selection. In the measured controlled benchmarks, speedups of roughly  $52\text{--}58\times$  over the stated baselines were obtained.

Experiment 206 used local 1-for-1 replacements and produced much larger end-to-end gains. The best measured speedups were approximately  $486\times$ ,  $677\times$ ,  $643\times$ , and  $372\times$  at 48, 54, 60, and 66 bits respectively.

Experiment 207 extended the neighborhood to 2-for-2 moves. The strongest recorded points were approximately:

- 48 bits: 0.002527550 s and  $487.95\times$  baseline;
- 54 bits: 0.000154645 s and  $685.53\times$ ;
- 60 bits: 0.000047340 s and  $715.70\times$ ;
- 66 bits: 0.000018910 s and  $398.46\times$ .

These results demonstrate non-additive prime interactions. The identity of the best set cannot be inferred by independently ranking primes.

Experiment 208 used beam search with survivor count as objective and found subsets with fewer survivors but worse runtime. Experiments 209–211 demonstrated that runtime calibration and representation switching can themselves consume enough time to erase theoretical filtering gains.

## 6.4 Representation benchmark

Experiment 212 isolated four representations. Table 2 gives the measured times.

Table 2: Measured representation performance from Experiment 212.

| Bits | Boolean     | Packed NumPy | Packed fast | Python integer |
|------|-------------|--------------|-------------|----------------|
| 48   | 0.002804201 | 0.0248095    | 0.0238616   | 0.11096        |
| 54   | 0.000201420 | 0.002159     | —           | 0.0003615      |
| 60   | 0.000055330 | 0.0006176    | —           | 0.00005738     |
| 66   | 0.000019865 | 0.0001431    | —           | 0.000007195    |

The result is that no representation dominates universally. Boolean arrays were strongest on several smaller explicit intervals, while Python integer bitsets became highly competitive at larger sizes.

## 7 Random Generalization

Experiments 213–215 showed that expanding the prime pool and optimizing for survivor count can lower the candidate count dramatically while making the actual program slower. A Pareto-frontier approach improved the trade-off in several instances.

Experiments 216–217 moved beyond the fixed benchmark suite by generating random semiprimes at 48–88 bits, while still constraining factor separation so that the Fermat interval remained experimentally manageable. On the latest clean run, the following core repeatedly appeared at 66 bits and above:

$$S = \{3, 5, 11, 17, 19, 31, 41, 43, 53, 59, 61\}. \quad (6)$$

Table 3: Latest controlled-random QR measurements.

| Bits | Set size | Survivors | Median time   | Speedup |
|------|----------|-----------|---------------|---------|
| 48   | 13       | 16        | 0.000096660 s | 1.094×  |
| 54   | 14       | 4         | 0.000046780 s | 1.045×  |
| 60   | 13       | 1         | 0.000010640 s | 1.866×  |
| 66   | 11       | 1         | 0.000004300 s | 1.000×  |
| 70   | 11       | 1         | 0.000003260 s | 1.000×  |
| 74   | 12       | 1         | 0.000007170 s | 1.315×  |
| 78   | 13       | 1         | 0.000012890 s | 6.659×  |
| 80   | 13       | 1         | 0.000006130 s | 1.675×  |
| 84   | 12       | 1         | 0.000004830 s | 1.462×  |
| 88   | 14       | 13        | 0.000153820 s | 13.724× |

The repeated appearance of (6) is an interesting empirical phenomenon. However, it should not be over-interpreted. These random instances were not uniformly distributed over all semiprimes because factor separation was deliberately restricted. The experiment therefore supports the existence of a recurring high-performing QR basis on the tested controlled distribution, but it does not prove universality or optimality.

## 8 Wide Independent Factors: Why the Explicit Interval Method Breaks

Experiments 218 and 219 were designed to remove the remaining near-square bias. The generator was changed so that  $p$  and  $q$  were independently sampled, subject only to a minimum relative separation such as

$$\frac{q-p}{p} \geq 0.20.$$

Experiment 218 initially combined this requirement with a maximum Fermat interval of 50 million during generation. That combination was internally incompatible at the target bit sizes, causing repeated generation failure.

Experiment 219 corrected the generator by separating instance generation from benchmarkability. A 48-bit sample, for example, was

$$p = 200960367359113, \quad q = 254705979343223,$$

with

$$\frac{q}{p} \approx 1.26744384$$

and a Fermat interval of

$$1,590,367,548,692.$$

Other generated cases produced intervals reaching many orders of magnitude beyond this.

The experiment therefore established a useful engineering fact: genuinely wide random semiprimes cannot be tested with a QR implementation that first materializes every  $x$  between  $\lceil \sqrt{n} \rceil$  and  $(p+q)/2$ . The method becomes memory-infeasible long before the underlying modular predicates cease to be mathematically valid.

Consequently, Experiments 218–219 should be treated as generator and scalability diagnostics rather than as positive or negative QR evidence. Their intended QR tests were skipped because the candidate interval was too large.

## 9 What the Experiments Establish

**Proposition 1** (QR necessity). *For any odd prime  $\ell$ , the true Fermat value  $x = (p + q)/2$  satisfies*

$$x^2 - n \in QR(\ell).$$

*Hence intersecting these QR conditions never discards the true solution.*

*Proof.* From  $x^2 - n = y^2$ , reduction modulo  $\ell$  gives  $x^2 - n \equiv y^2 \pmod{\ell}$ , which is a quadratic residue. □

**Observation 1** (Runtime is not survivor count). *The experiments repeatedly found subsets with fewer survivors but larger wall-clock time. Therefore survivor count is an incomplete objective for QR-prime selection.*

**Observation 2** (Representation matters). *Boolean arrays, packed arrays, and Python integer bitsets showed different performance regimes. The optimal representation depends on interval length and implementation overhead.*

**Observation 3** (Factor separation dominates Fermat interval size). *Equation (2) shows directly why independently sampled, widely separated factors lead to enormous Fermat intervals even when the bit size is still below 100 bits.*

## 10 What Was Ruled Out or Weakened

The experiments provide several negative results that are as important as the successful QR sieve:

1. A universal low-rank error model was not observed.
2. A universally singular carry matrix was not observed.
3. AC-3 and related local consistency did not collapse the global residue CSP at higher sizes.
4. Multiplicative rank-1 behavior in the  $C$  representation was shown to have tautological degrees of freedom.
5. Pure residue search does not scale adequately in the tested formulation.
6. Adding more QR primes to force the survivor count toward one can make the computation slower rather than faster.
7. Uniformly random wide factors invalidate the explicit full-interval array representation used by the earlier QR implementation.

## 11 Current Best Interpretation

The most strongly supported conclusion is the following:

Selected small quadratic-residue moduli can form a highly effective practical sieve for Fermat factorization when the Fermat interval is computationally manageable. The modulus-selection problem is itself an empirical optimization problem governed by filtering strength, representation cost, and higher-order interactions among primes. A recurring high-performing prime core appears across several controlled random experiments, but current evidence does not establish that this core is universal or theoretically optimal.

This conclusion is intentionally narrower than a claim of a new factoring algorithm. It is supported directly by the measured experiments and remains compatible with the negative results from the CRT/CSP line.

## 12 Methodological Lessons

The research produced a number of reusable principles for future experiments.

**Separate mathematical validity from implementation feasibility.** A modular predicate can remain perfectly valid after explicit interval enumeration becomes impossible. Experiments should therefore avoid conflating “cannot materialize the candidate set” with “the filter does not work”

**Measure end-to-end runtime.** A smaller candidate set is not inherently a faster algorithm. Precomputation, mask construction, conversions, and memory traffic can dominate.

**Stress-test structural relations for independence.** If arbitrary residue vectors can reproduce a claimed relation, that relation is not independent information about the hidden factors.

**Change one major assumption at a time.** The failed first version of Experiment 218 combined a wide-factor constraint with an incompatible interval bound. Separating generation from benchmarkability fixed the engineering mistake immediately.

## 13 Recommended Next Experimental Direction

The next generalized QR experiment should not allocate the entire Fermat interval. Instead, one should work with residue classes or streaming blocks. A mathematically natural direction is to select a modulus

$$L = \prod_{\ell \in S} \ell$$

(or a compatible product of prime powers), derive the surviving congruence classes of  $x \pmod{L}$ , and then enumerate only those arithmetic progressions that intersect the Fermat interval.

The key experimental variable should then be factor separation, not merely bit length. A fixed QR basis can be tested on semiprimes with the same bit length but systematically varied  $q/p$ , allowing the research to disentangle:

1. the filtering power of the QR basis;
2. the number of admissible residue classes per CRT period;
3. the cost of enumerating those classes;
4. the dependence of the total work on Fermat interval length.

Such an experiment would directly address the limitation exposed by Experiment 219 without changing the mathematical hypothesis being tested.

## 14 Conclusion

The research began with attempts to expose hidden prime factors through CRT, radix, carry, matrix-rank, and constraint-satisfaction structure. Several promising-looking relations were shown to be either non-universal or tautological. The investigation then shifted toward a more modest but experimentally strong idea: quadratic-residue filtering of Fermat candidates.

The QR line produced the clearest practical gains. Adaptive selection and local neighborhood search found prime sets that reduced real end-to-end runtime by hundreds of times on the fixed manageable intervals, and controlled random experiments revealed a recurring small-prime core at higher bit sizes. At the same time, the experiments made clear that survivor count alone is not an adequate objective and that the entire representation and enumeration pipeline must be optimized.

The final wide-factor experiment was valuable precisely because it failed for the right reason. It removed the artificial near-square assumption and exposed the real scalability barrier: explicit Fermat interval enumeration. The next stage should therefore retain the QR mathematics while replacing full interval materialization with congruence-class or streaming enumeration.